

README / Setup Guide

Overview

This document provides a comprehensive guide for setting up and running the employee leave application for our organization. The application allows employees to submit leave requests, managers to approve or reject these requests, and HR to monitor the overall leave status of employees. The core benefits of this application include streamlined leave management, improved transparency, and reduced administrative overhead. The target users are employees, managers, and HR personnel within the organization.

Key Features

- Employees can submit leave requests with details such as leave type, start date, end date, and reason.
- Managers can approve or reject leave requests and view the leave status of their team members.
- HR personnel can monitor the overall leave status of all employees and generate reports.

Prerequisites

- Python 3.9+
- Node.js 16+
- PostgreSQL 13+
- Docker and Docker Compose
- A Unix-like operating system (Linux or macOS) or Windows Subsystem for Linux (WSL)

Project Structure

The repository layout is as follows:

- `backend/`: Contains the backend server code, including API endpoints, business logic, and database interactions. Key frameworks include Flask for the web server and SQLAlchemy for ORM.
- `frontend/`: Contains the frontend code, including React components, state management, and API calls. Key frameworks include React, Redux, and Axios.
- `shared/`: Contains shared utilities and helper functions used by both the backend and frontend.
- `docs/`: Contains documentation, including API documentation and user guides.

Installation & Setup

Backend Setup

1. Clone the repository: `git clone <url>`
2. Navigate to the project: `cd project-name`
3. Create virtual environment: `python -m venv venv`
4. Activate: `source venv/bin/activate` (Linux/Mac) or `venv\Scripts\activate` (Windows)
5. Install dependencies: `pip install -r requirements.txt`
6. Create a `.env` file in the backend directory with the following content:

```
SECRET_KEY=your_secret_key
SQLALCHEMY_DATABASE_URI=postgresql://username:password@localhost:5432/leave_db
```

Frontend Setup

1. Navigate to frontend: `cd frontend`
2. Install dependencies: `npm install`
3. Build/prepare assets: `npm run build`

Configuration

Required Environment Variables

- `SECRET_KEY`: Controls the secret key used for session management and CSRF protection. Format: string, Example value: `your_secret_key`.
- `SQLALCHEMY_DATABASE_URI`: Controls the database connection string. Format: string, Example value: `postgresql://username:password@localhost:5432/leave_db`.

Optional Environment Variables

- `PORT` (default: 5000): Controls the port on which the backend server runs.

Running the Application

Starting the Backend

1. Ensure virtual environment is activated
2. Set environment variables: `export VAR=value`
3. Start server: `python app.py`
4. Verify: Backend is running on `http://localhost:5000`

Starting the Frontend

1. Navigate to frontend directory: `cd frontend`
2. Start dev server: `npm run dev`
3. Open browser: `http://localhost:3000` (or displayed URL)
4. Verify: UI loads and connects to backend

Verification Checklist

- ☐ Backend health check responds at GET `/api/health` with 200 OK
- ☐ Frontend loads without console errors
- ☐ Basic workflow completes end-to-end (e.g., employee submits a leave request and manager approves it)
- ☐ API endpoints respond with expected data format

Troubleshooting

Backend Issues

- **Port already in use:** Change PORT env var or kill process on port 5000
- **Module not found:** Run `pip install -r requirements.txt` again
- **Database connection error:** Verify connection string in .env, check service is running
- **Authentication error:** Verify API keys/tokens in .env are correct and not expired

Frontend Issues

- **Blank page or won't load:** Check browser console for errors, ensure backend is running
- **API connection error:** Verify backend URL in config, check CORS settings
- **Module resolution error:** Delete `node_modules` and run `npm install` again
- **Build fails:** Ensure Node.js version matches requirements, clear cache: `npm cache clean --force`

Common Workflows

Employee Submits Leave Request

1. Employee logs in and navigates to the leave request page.
2. Employee fills out the leave request form with details such as leave type, start date, end date, and reason.
3. Employee submits the form.
4. The leave request is sent to the manager for approval.

Manager Approves/Rejects Leave Request

1. Manager logs in and navigates to the leave requests page.
2. Manager views the pending leave requests.
3. Manager selects a leave request and approves or rejects it.
4. The employee is notified of the approval or rejection via email.

Deployment Notes

- Ensure secrets management is handled securely, e.g., using environment variables or secret management tools.
- Configure scaling for the backend server to handle increased load during peak usage times.
- Set up a CI/CD pipeline for automated testing and deployment.

Getting Help

- Check logs: Backend logs in console, frontend logs in browser DevTools
- Review error message and stack trace for clues
- Check environment variables are set correctly
- Refer to the API Documentation and User Guide for more details